- Opérateurs en OCaml/Python :

	OCaml	Python
test d'égalité	`=`	`==`
test de différence	`<>`	`!=`
division euclidienne	`/`	`//`
modulo	`mod`	`%`
et	`&&`	`and`
ou	`||`	`or`

- Références :

	OCaml
Définition	`let r = ref ...`
Accéder à la valeur	`!r`
Modifier la valeur	`r := ...`

- Décomposer un `tuple` : `let a, b, ... = p`.
- Définir une fonction : `let f x y = ...` ou `fun x y -> ...`
- On créé souvent une liste avec une fonction récursive :

```
let rec range n = (* renvoie [n - 1; ...; 1; 0] *)
    if n = 0 then []
    else (n-1)::range (n - 1)
```

On peut éventuellement utiliser une référence sur une liste :

```
let range n =
    let l = ref [] in (* moins idiomatique *)
    for i = n - 1 downto 0 do
        l := i::!l
    done;
    !l
```

- **Impossible de modifier une liste** `l` : `e::l` renvoie une **nouvelle liste** mais ne modifie pas `l`.
  `let l = [] in ...` ne sert à rien.
- **Impossible d'écrire** `l.(i)` pour une liste `l` : il faut utiliser une fonction récursive pour parcourir une liste.
  Exemple :

```
let rec appartient e l = match l with
    | [] -> false
    | t::q -> e = t || appartient e q
```

- Chaque cas d'un `match` sert à **définir de nouvelles variables**, et ne permet pas de comparer des valeurs.
  Exemple : Dans `appartient`, il ne faut pas écrire `| e::q -> ...` (ceci remplacerait le `e` en argument).
  Pour tester l'égalité : `| t::q -> if t = e then ...` ou `| t::q when t = e -> ...`.
- Les indices d'un tableau à n éléments vont de 0 à $n-1$:

```
let appartient e t =
    let r = ref false in
    for i = 0 to  n - 1 do
        if t.(i) = e then r := true
    done;
    !r
```

Cette fonction est de type `'a -> 'a array -> bool`.

- **Impossible de renvoyer une valeur à l'intérieur d'une boucle**:

```
let appartient e t =
    for i = 0 to n - 1 do
        if t.(i) = e then true (* NE MARCHE PAS !!! *)
    done;
    false
```

- Types union et enregistrement :

```
(* type union : l'un OU l'autre *)
type 'a option = None | Some of 'a
(* on utilise match (et fonction récursive)
   sur les types union *)
let add x  y = match x, y with
    | None, _ | _, None -> None
    | Some x1, Some y1 -> Some (x1 + y1)

(* type enregistrement : l'un ET l'autre *)
type fraction = { num : int; denum : int }
let f = { num = 1; denum = 4 }
f.denum (* donne 4 *)
```

- Quelques fonctions utiles (avec les équivalents `Array.map`...):
  - `Array.make_matrix n p e` renvoie une matrice $n \times p$ dont chaque élément est `e`
  - `List.iter f l` appelle `f` sur chaque élément de `l`
  - `List.map f [a1; ...; an]` renvoie `[f a1; ...; f an]`
  - `List.filter f l` renvoie la liste des éléments `e` tels que `f e`
  - `List.exists f l` et `List.for_all f l`.

Type	Exemple	Obtenir valeur	Modification	Taille	Création
`array`	`[|1; 2|] : int array`	`t.(i)`	`t.(i) <- ...`	`Array.length` (en O(1))	`Array.make n e` (en O(n))
`string`	`"abc" : string`	`s.[i]`		`String.make n e` (en O(n))	`String.make n e`
`list`	`[1; 2] : int list`	Fonction récursive		`List.length` (en O(n))	
`tuple`	`(1, "abc", [1; 2]) : int*string*int list`	`let a, b, c = t`			

Contrairement aux `list`, `array` et `string`, un `tuple` (n-uplet) peut contenir des types différents.

- Opérateurs en OCaml/Python :

	OCaml	Python
test d'égalité	`=`	`==`
test de différence	`<>`	`!=`
division euclidienne	`/`	`//`
modulo	`mod`	`%`
et	`&&`	`and`
ou	`||`	`or`

- Références :

	OCaml
Définition	`let r = ref ...`
Accéder à la valeur	`!r`
Modifier la valeur	`r := ...`

- Décomposer un `tuple` : `let a, b, ... = p`.
- Définir une fonction : `let f x y = ...` ou `fun x y -> ...`
- On créé souvent une liste avec une fonction récursive :

```
let rec range n = (* renvoie [n - 1; ...; 1; 0] *)
    if n = 0 then []
    else (n-1)::range (n - 1)
```

On peut éventuellement utiliser une référence sur une liste :

```
let range n =
    let l = ref [] in (* moins idiomatique *)
    for i = n - 1 downto 0 do
        l := i::!l
    done;
    !l
```

- **Impossible de modifier une liste** `l` : `e::l` renvoie une **nouvelle liste** mais ne modifie pas `l`.
  `let l = [] in ...` ne sert à rien.
- **Impossible d'écrire** `l.(i)` pour une liste `l` : il faut utiliser une fonction récursive pour parcourir une liste.
  Exemple :

```
let rec appartient e l = match l with
    | [] -> false
    | t::q -> e = t || appartient e q
```

- Chaque cas d'un `match` sert à **définir de nouvelles variables**, et ne permet pas de comparer des valeurs.
  Exemple : Dans `appartient`, il ne faut pas écrire `| e::q -> ...` (ceci remplacerait le `e` en argument).
  Pour tester l'égalité : `| t::q -> if t = e then ...` ou `| t::q when t = e -> ...`.
- Les indices d'un tableau à n éléments vont de 0 à $n-1$:

```
let appartient e t =
    let r = ref false in
    for i = 0 to  n - 1 do
        if t.(i) = e then r := true
    done;
    !r
```

Cette fonction est de type `'a -> 'a array -> bool`.

- **Impossible de renvoyer une valeur à l'intérieur d'une boucle**:

```
let appartient e t =
    for i = 0 to n - 1 do
        if t.(i) = e then true (* NE MARCHE PAS !!! *)
    done;
    false
```

- Types union et enregistrement :

```
(* type union : l'un OU l'autre *)
type 'a option = None | Some of 'a
(* on utilise match (et fonction récursive)
   sur les types union *)
let add x  y = match x, y with
    | None, _ | _, None -> None
    | Some x1, Some y1 -> Some (x1 + y1)

(* type enregistrement : l'un ET l'autre *)
type fraction = { num : int; denum : int }
let f = { num = 1; denum = 4 }
f.denum (* donne 4 *)
```

- Quelques fonctions utiles (avec les équivalents `Array.map`...):
  - `Array.make_matrix n p e` renvoie une matrice $n \times p$ dont chaque élément est `e`
  - `List.iter f l` appelle `f` sur chaque élément de `l`
  - `List.map f [a1; ...; an]` renvoie `[f a1; ...; f an]`
  - `List.filter f l` renvoie la liste des éléments `e` tels que `f e`
  - `List.exists f l` et `List.for_all f l`.

Type	Exemple	Obtenir valeur	Modification	Taille	Création
`array`	`[|1; 2|] : int array`	`t.(i)`	`t.(i) <- ...`	`Array.length` (en $O(1)$)	`Array.make n e` (en $O(n)$)
`string`	`"abc" : string`	`s.[i]`		`String.make n e` (en $O(n)$)	`String.make n e`
`list`	`[1; 2] : int list`	Fonction récursive		`List.length` (en $O(n)$)	
`tuple`	`(1, "abc", [1; 2]) : int*string*int list`	`let a, b, c = t`			

Contrairement aux `list`, `array` et `string`, un `tuple` (n-uplet) peut contenir des types différents.

Définitions

- Un **graphe non orienté** est un couple $G = (V, E)$ où V est un ensemble fini de **sommets** et E est un ensemble d'**arêtes**, chaque arête étant un ensemble de deux sommets de V.
  Si $e = \{u, v\} \in E$, on dit que u et v sont **adjacents** ou **voisins** et que ce sont les extrémités de e.
  Le **degré** $\deg(v)$ d'un sommet v est son nombre de voisins.
  Une **feuille** est un sommet de degré 1.
- Un **graphe orienté** est la même chose qu'un graphe non orienté, sauf que chaque arête est un couple (u, v) (représenté par $u \longrightarrow v$) au lieu d'un ensemble.
- Si G est un graphe non orienté à n sommets et p arêtes alors

$$p \leq \binom{n}{2} = O(n^2)$$

- (HP) Si $G = (V, E)$ est un graphe non orienté alors :

$$\sum_{v \in V} \deg(v) = 2|E|$$

> Preuve : récurrence sur le nombre d'arêtes ou double comptage ($\sum_{v \in V} \deg(v)$ compte deux fois chaque arête, puisqu'une arête a deux extrémités).

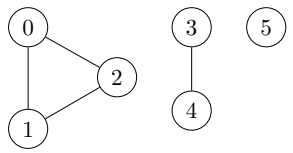
- (HP) Un graphe possède un nombre pair de sommets de degrés impairs.

> Preuve : $\underbrace{\sum_{\deg(v) \text{ pair}} \deg(v)}_{\text{pair}} + \sum_{\deg(v) \text{ impair}} \deg(v) = \underbrace{2|E|}_{\text{pair}}$

- Un **chemin** est une suite d'arêtes consécutives différentes.
  La **longueur** d'un chemin est son nombre d'arêtes.
  La **distance** $d(u, v)$ de u à v est la plus petite longueur d'un chemin de u à v (∞ si il n'y a pas de chemin) : c'est une distance au sens mathématique.

Représentations

- Exemple de graphe avec ses représentations en OCaml :



Matrice d'adjacence

```
[|[|0; 1; 1; 0; 0; 0|];
[|1; 0; 1; 0; 0; 0|];
[|1; 1; 0; 0; 0; 0|];
[|0; 0; 0; 0; 1; 0|];
[|0; 0; 0; 1; 0; 0|];
[|0; 0; 0; 0; 0; 0|]|]
```

Remarque : une matrice adjacence d'un graphe non orienté est toujours symétrique.

Liste d'adjacence

```
[|[1; 2];
[0; 2];
[0; 1];
[4];
[3];
[]|]
```

- Pour un graphe orienté à n sommets et p arêtes :

	Matrice d'adjacence	Liste d'adjacence
complexité mémoire	$O(n^2)$	$\boxed{O(n+p)}$
tester si $\{u, v\} \in E$	$\boxed{O(1)}$	$O(\deg(u))$
parcourir voisins de u	$O(n)$	$\boxed{O(\deg(u))}$

On utilise plutôt une liste d'adjacence pour un graphe avec peu d'arêtes et une matrice d'adjacence sinon.

Connexité, cycle

- Un graphe G non orienté est **connexe** si, pour tout sommets u et v, il existe un chemin entre u et v dans G.
- Un graphe G orienté est **fortement connexe** si, pour tout sommets u et v, il existe un chemin de u à v dans G.
- (HP) Un graphe connexe à n sommets a au moins $n - 1$ arêtes.

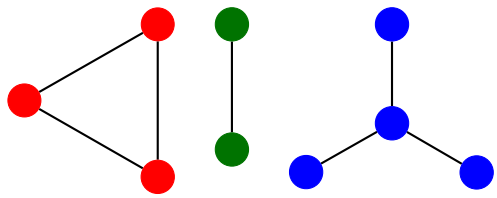
> Preuve : Soit $\mathcal{H}(n)$: « un graphe connexe à n sommets a au moins $n - 1$ arêtes ».
> $\mathcal{H}(1)$ est vraie car un graphe à 1 sommet possède 0 arête.
> Supposons $\mathcal{H}(n)$. Soit $G = (V, E)$ un graphe connexe à $n + 1$ sommets.
> - Si G a un sommet v de degré 1 alors $G - v$ (G auquel on a enlevé le sommet v) est un graphe *connexe* à n sommets donc, par $\mathcal{H}(n)$, $G - v$ a au moins $n - 1$ arêtes. Donc G a au moins n arêtes.
> - Sinon, tous les sommets de G sont de degré ≥ 2.
> Alors $2|E| = \sum_{v \in V} \deg(v) \geq 2(n + 1) \geq 2n$.
> Donc $|E| \geq n$, ce qui montre $\mathcal{H}(n + 1)$.

- La relation suivante est une relation d'équivalence sur un graphe non orienté :

$$u \sim v \iff \text{il existe un chemin entre } u \text{ et } v$$

Les classes d'équivalences pour $\sim$ sont les sous-graphes connexes maximaux (au sens de $\subseteq$) de G, ils sont appelés **composantes connexes**.

> Exemple : un graphe avec 3 composantes connexes.



Remarque : Un graphe est connexe ssi il contient une unique composante connexe.

- (HP) Un graphe acyclique contient un sommet de degré au plus 1. Preuve : Supposons que tous les sommets soient de degrés ≥ 2.
  Faisons partir un chemin depuis un sommet quelconque en visitant à chaque étape le sommet adjacent différent du prédécesseur (possible car les degrés sont ≥ 2). Comme le

nombre de sommets est fini, ce chemin revient nécessairement sur un sommet déjà visité, ce qui donne un cycle.

- (HP) Un graphe acyclique à n sommets possède au plus $n-1$ arêtes.

  Preuve : Montrons par récurrence $\mathcal{H}(n)$: « un graphe acyclique à n sommets a au plus $n-1$ arêtes ».
  $\mathcal{H}(1)$ est vraie car un graphe à 1 sommet possède 0 arête.
  Supposons $\mathcal{H}(n)$. D'après le lemme, un graphe G acyclique à $n+1$ sommets possède un sommet v de degré ≤ 1.
  Comme G est acyclique, $G-v$ l'est aussi et a au plus $n-1$ arêtes, par $\mathcal{H}(n)$.
  Donc G a au plus $n-1+\deg(v) \leq n$ arêtes, ce qui montre $\mathcal{H}(n+1)$.

- Un graphe T à n sommets et p arêtes est un **arbre** s'il vérifie l'une des conditions équivalentes :
  1. T est connexe acyclique (définition).
  2. T est connexe et $p = n-1$.
  3. T est acyclique et $p = n-1$.

  Remarque : les arbres vus en 1ère année étaient enracinés. Ici il n'y a pas de racine.
  Preuve :
  $1 \implies 2$: Si T est connexe et acyclique alors $p \geq n-1$ (connexe) et $p \leq n-1$ (acyclique) donc $p = n-1$.
  $2 \implies 3$: Supposons T connexe et $p = n-1$.
  Par l'absurde, supposons que T contienne un cycle C. Soit e une arête de C.
  Montrons alors que $T-e$ est connexe.
  Soient donc u et v deux sommets de T. Comme T est connexe, il existe un chemin C' dans T entre u et v.
  Si C' ne contient pas e, C' est un chemin de u à v dans T.
  Si C' contient e, remplaçons e par le reste de C. C' est alors un chemin entre u et v.
  $T-e$ est donc connexe, ce qui contredit le fait que $T-e$ contienne n sommets et $n-2$ arêtes.
  $3 \implies 1$: Supposons T acyclique et $p = n-1$.
  Supposons par l'absurde que T ne soit pas connexe.
  Alors T a $k \geq 2$ composantes connexes T_1, ..., T_k. Notons n_i et p_i le nombre de sommets et d'arêtes de T_i.
  Comme T_i est connexe, $p_i \geq n_i - 1$. Donc $p = \sum p_i \geq \sum (n_i - 1) = n - k$, ce qui est absurde avec l'hypothèse $p = n-1$.

Parcours de graphe

- **Parcours en profondeur** : on visite les sommets le plus profondément possible avant de revenir en arrière.

```
let dfs (g : int list array) r =
    let n = Array.length g in
    let visited = Array.make n false in
    let rec aux v =
        if not visited.(v) then (
            visited.(v) <- true;
            (* traiter v *)
            List.iter aux g.(v)
        ) in
    aux r
```

Complexité avec représentation par liste d'adjacence :
Soit n le nombre de sommets et p le nombre d'arêtes.
`Array.make n false` est en $\mathrm{O}(n)$.
Chaque arête est parcourue au plus une fois, d'où $\mathrm{O}(p)$ appels récursifs de `aux`.
Au total : $\boxed{\mathrm{O}(n+p)}$.
Remarque : La complexité est $\mathrm{O}(n^2)$ avec une matrice d'adjacence.

- Application : recherche d'un cycle dans un graphe non orienté, en faisant attention à ne pas considérer une arête comme cycle (pas d'appel récursif sur le père).

```
let has_cycle (g : int list array) =
  let n = Array.length g in
  let visited = Array.make n false in
  let ans = ref false in
  let rec aux p u = (* p a permis de découvrir u *)
    if not visited.(u) then (
      visited.(u) <- true;
      List.filter ((<>) p) g.(u)
      |> List.iter (aux u)
    )
    else ans := true in
  for i = 0 to n - 1 do
    if not visited.(i) then aux i i
  done;
  !ans
```

- **Parcours en largeur** : on visite les sommets par distance croissante depuis le sommet de départ. Pour cela, on stocke les prochains sommets à visiter dans une file `q` :

```
let bfs (g : int list array) r =
    let seen = Array.make (Array.length g) false in
    let q = Queue.create () in
    let add v =
        if not seen.(v) then (
            seen.(v) <- true; Queue.add v q
        ) in
    add r;
    while not (Queue.is_empty q) do
        let v = Queue.pop q in
        (* traiter v *)
        List.iter add g.(v)
    done
```

- Application : calcul de distance (en nombre d'arêtes), en stockant des couples (sommet, distance) dans `q`.

```
let bfs g r =
    let dist = Array.make (Array.length g) (-1) in
    let q = Queue.create () in
    let add d v =
        if dist.(v) = -1 then (
            dist.(v) <- d;
            Queue.add (v, d) q
        ) in
    add 0 r;
    while not (Queue.is_empty q) do
        let u, d = Queue.pop q in
        List.iter (add (d + 1)) g.(u)
    done;
    dist (* dist.(u) est la distance de r à u *)
```

- Pour obtenir les plus courts chemins, on peut stocker le sommet `pred.(v)` qui a permis de découvrir `v` :

```
let bfs g r =
    let pred = Array.make (Array.length g) (-1) in
    let q = Queue.create () in
    let add p v = (* p est le père de v *)
        if pred.(v) = -1 then (pred.(v) <- p; Queue.add v q)
    add r r;
    while not (Queue.is_empty q) do
        let v = Queue.pop q in
        List.iter (add v) g.(v)
    done;
    pred (* pred.(v) = père de v dans le parcours *)
```

On peut alors obtenir un plus court chemin de `r` à `v` en remontant les pères de `v` jusqu'à obtenir `r` :

```
let rec path pred v =
    if pred.(v) = v then [v]
    else v::path pred pred.(v)
```

File de priorité

- Une **file de priorité max** (FP max) est une structure de données possédant les opérations suivantes :
  - Extraire maximum : supprime et renvoie le maximum
  - Ajouter élément
  - Tester si la FP est vide

  On définit une FP min en remplaçant maximum par minimum.

- Il est possible d'utiliser un arbre binaire de recherche pour implémenter une FP max, en utilisant le fait que le maximum est tout à droite de l'arbre. Les opérations sont d'ajout et d'extraction sont alors linéaires en la hauteur de l'arbre :

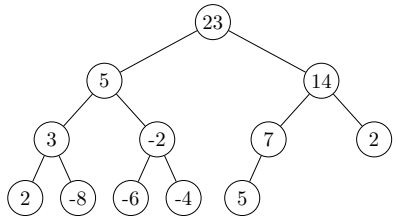
```
type 'a abr = V | N of 'a * 'a abr * 'a abr

(* extraire a renvoie (max de a, reste de a) *)
let rec extraire = function
    | V -> failwith "FP vide"
    | N(r, g, V) -> (r, g)
    | N(r, g, d) ->
        let (m, d') = extraire_max d in
        (m, N(r, g, d'))

let rec ajoute e = function
    | V -> N(e, V, V)
    | N(r, g, d) ->
        if e > r then N(e, g, d)
        else if e < r then N(r, ajoute e g, d)
        else N(r, g, d)
```

  On peut aussi mettre à jour un élément (changer sa valeur) en le supprimant puis en le réajoutant (avec la nouvelle valeur).

Tas

- Un **tas max** est un arbre binaire presque complet (tous les niveaux, sauf le dernier, sont complets) où chaque noeud est plus grand que ses fils.
  Remarque : la racine est le maximum du tas.
  Un tas min est comme un tas max, sauf que chaque noeud doit être plus petit que ses fils.

  Exemple :



- Soit T un arbre binaire presque complet de hauteur h et avec n noeuds. Alors $h = \lfloor \log_2(n) \rfloor$ (donc $\boxed{h = \mathrm{O}(\log(n))}$).

  Preuve : T contient plus de sommets qu'un arbre binaire complet de hauteur $h-1$ et moins de sommets qu'un arbre binaire complet de hauteur h, donc :

$$\begin{aligned} & 2^h - 1 < n \leq 2^{h+1} - 1 \\ \implies & 2^h \leq n < 2^{h+1} \\ \implies & h \leq \log_2(n) < h+1 \\ \implies & h = \lfloor \log_2(n) \rfloor \end{aligned}$$

- On stocke les noeuds du tas dans un tableau `t` tel que :
  - `t.(0)` est la racine de `t`.
  - `t.(i)` a pour fils `t.(2*i + 1)` et `t.(2*i + 2)`, si ceux-ci sont définis.

  Le père de `t.(j)` est donc `t.((j - 1)/2)` (si $j \neq 0$).
  Exemple : le tas en exemple ci-dessus est représenté par `t = [|23; 5; 14; 3; -2; 7; 2; 2; -8; -6; -4; 5|]`.

- En pratique, comme on ne peut pas ajouter d'élément à un tableau, on utilise un tableau `t` plus grand que le nombre d'éléments du tas pour pouvoir y ajouter des éléments ensuite et on stocke le nombre d'éléments dans une variable `n` :

```
type 'a tas = {t : 'a array; mutable n : int}

let swap tas i j = (* échange tas.t.(i) et tas.t.(j) *)
    let tmp = tas.t.(i) in
    tas.t.(i) <- tas.t.(j);
    tas.t.(j) <- tmp
```

- On utilise deux fonctions auxiliaires pour rétablir la propriété de tas après modification :
  - `up tas i` : suppose que `tas` est un tas max sauf `tas.t.(i)` qui peut être supérieur à son père. Fait monter (on dit aussi percoler) `tas.t.(i)` de façon à obtenir un tas max.
  - `down tas i` : suppose que `tas` est un tas max sauf `tas.t.(i)` qui peut être inférieur à un fils. Fait descendre `tas.t.(i)` de façon à obtenir un tas max.

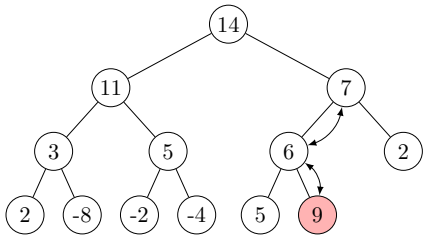
```
let rec up h i =
    let p = (i - 1)/2 in
    if i <> 0 && tas.t.(p) < tas.t.(i) then (
        swap h i p;
        up h p
    )

let rec down tas i =
  let m = ref i in (* maximum parmi i et ses fils *)
  if 2*i + 1 < tas.n && tas.t.(2*i + 1) > tas.t.(!m)
  then m := 2*i + 1;
  if 2*i + 2 < tas.n && tas.t.(2*i + 2) > tas.t.(!m)
  then m := 2*i + 2;
  if !m <> i then (
    swap h i !m;
    down h !m
  )
```

- Pour ajouter un élément à un tas : l'ajouter comme dernière

feuille (dernier indice du tableau) puis faire remonter.
Complexité : $O(h) = \boxed{O(\log(n))}$.

```
let ajoute tas e =
    tas.t.(tas.n) <- e;
    up h tas.n;
    tas.n <- tas.n + 1
```

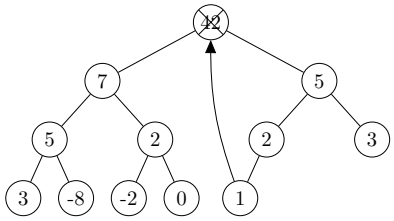
Exemple : ajout de 9 et percolation sur un exemple.



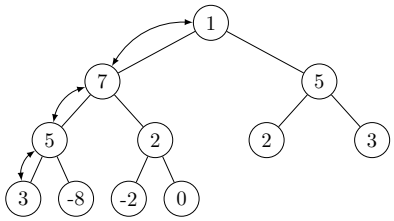
- Pour extraire le maximum d'un tas : échanger la racine avec la dernière feuille puis descendre la nouvelle racine.
  Complexité : $O(h) = \boxed{O(\log(n))}$.

```
let rec extract_max h =
    swap h 0 (h.n - 1);
    h.n <- h.n - 1;
    down h 0;
    h.a.(h.n)
```

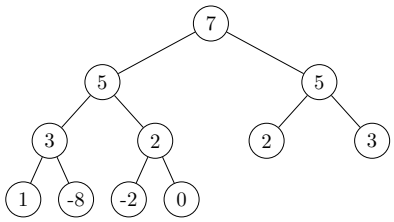
Exemple :



Suppression de la racine qu'on remplace par la dernière feuille



Percolation



Tas obtenu après extraction du maximum

- Parfois, on peut vouloir mettre à jour un élément dans un tas. Pour cela, on peut

Applications

- **Tri avec une file de priorité** : on ajoute tous les éléments dans un tas puis on les extrait un par un.
  On obtient ainsi le **tri par tas** en $O(n\log(n))$ avec un tas. On peut même utiliser le tableau en entrée comme tableau `tas.t` du tas, ce qui permet d'obtenir un tri en place, c'est-à-dire sans utiliser de tableau supplémentaire ($O(1)$ en mémoire).
- **Algorithme de Dijkstra**, pour extraire le sommet de distance estimée minimum à chaque itération. Dans le pseudocode suivant, on peut utiliser une file de priorité pour `q` :

```
Initialement : q contient tous les sommets
               dist.(s) <- 0
               dist.(v) <- infini, si v <> s

Tant que q est non vide:
    Extraire u de q tel que dist.(u) soit minimum
    Pour tout voisin v de u:
        Si dist.(u) + w(u, v) < dist.(v):
            dist.(v) <- dist.(u) + w(u, v)
```

- **Algorithme de Kruskal**, pour obtenir l'arête de poids minimum à chaque itération.

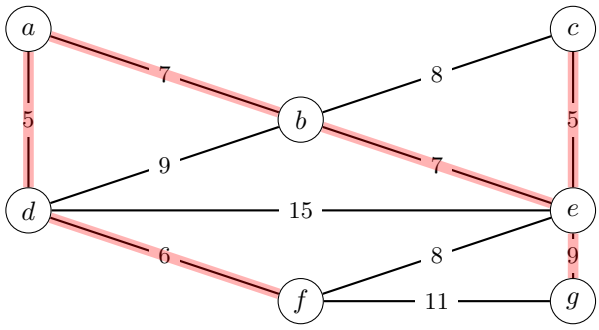
- Soit $G = (V, E)$ un graphe pondéré par $w : E \longrightarrow \mathbb{R}$.
  - Un **arbre couvrant** T de G est un ensemble d'arêtes de G qui forme un arbre et qui contient tous les sommets.
  - Son **poids** $w(T)$ est la somme des poids des arêtes de T.
  - Un arbre couvrant dont le poids est le plus petit possible est appelé un **arbre couvrant de poids minimum**.
- Soit G un graphe connexe pondéré par w.
  Alors G possède un arbre couvrant de poids minimum.

  Preuve : Soit $E = \{w(T) \mid T \text{ est un arbre couvrant de } G\}$.
  - $E \neq \emptyset$: l'ensemble des arêtes parcourues dans un parcours de graphe est un arbre couvrant.
  - E est fini.

  Donc E admet bien un minimum.

  Remarque : il n'y a pas forcément unicité d'un arbre couvrant de poids minimum.
- L'algorithme de **Kruskal** est un algorithme glouton permettant d'obtenir un arbre couvrant de poids minimum en sélectionnant les arêtes par poids croissant qui ne créent pas de cycle.

  Exemple : Un graphe avec un arbre couvrant de poids minimum obtenu par l'algorithme de Kruskal (en choisissant, dans l'ordre : ad, ec, df, ab, be, eg).



- L'algorithme de Kruskal renvoie bien un arbre T qui est couvrant et de poids minimum.

  Preuve : Montrons d'abord que T est un arbre couvrant.

  1. T est acyclique : d'après les choix de l'algorithme.
  2. T est connexe et couvrant : soient u et v deux sommets de G. Soit U l'ensemble des sommets accessibles depuis u dans T.
     Supposons $v \notin U$. Comme G est connexe, il existe une arête de G entre U et $V \setminus U$.
     Cette arête aurait dû être ajoutée à T, puisqu'elle ne créée pas de cycle.
     Contradiction: v est donc accessible depuis u dans T.
     Comme c'est vrai pour tout u, v, T est connexe.

  Montrons maintenant que T est de poids minimum.
  Soient T l'arbre obtenu par Kruskal et T^* un arbre de poids minimum.
  Si $T = T^*$, le théorème est démontré.
  Sinon, soit $e^* = \{u, v\} \in T^* \setminus T$. $T^* - e^*$ n'est pas connexe donc possède deux composantes connexes T^*_u (contenant u) et T^*_v (contenant v).
  Comme T est connexe, il existe un chemin C dans T reliant u et v. Ce chemin possède une arête e entre T^*_u et T^*_v.
  Soit $T^*_2 = T^* - e^* + e$. T^*_2 possède $n - 1$ arêtes et une seule composante connexe : c'est un arbre couvrant.
  De plus, $w(e) \leq w(e^*)$ sinon e^* aurait été ajouté avant e dans l'algorithme de Kruskal. On a donc $w(T^*) \geq w(T^*_2)$.
  On répète ce processus avec T^*_2 au lieu de T^*, ce qui nous donne des arbres couvrants T^*_3, T^*_4... jusqu'à obtenir T:

  $$w(T^*) \geq w(T^*_2) \geq w(T_3) \geq ... \geq w(T)$$

  $w(T) \leq w(T^*)$ montre que T est de poids minimum.
- Pour l'implémentation, on suppose l'existence d'une fonction `tri_aretes` qui renvoie la liste des arêtes triées par poids croissant (chaque arête étant un triplet (w, u, v) où w est le poids de l'arête $\{u, v\}$).

```
let chemin (t : int list array) u v =
(* détermine s'il existe un chemin de u à v dans t *)
(* on fait un parcours en profondeur depuis u *)
  let n = Array.length t in
  let visited = Array.make n false in
  let rec aux u =
    if not visited.(u) then (
       visited.(u) <- true;
       List.iter aux t.(u)
    ) in
   aux u;
   visited.(v)

let kruskal (g : int list array) =
    let n = Array.length g in
    let t = Array.make n [] in
    List.iter (fun (w, u, v) ->
        if not (chemin t u v) then (
            t.(u) <- v :: t.(u);
            t.(v) <- u :: t.(v);
    ) tri_aretes g;
    done; t
```

Remarque : on peut aussi utiliser une file de priorité au lieu du tri
Complexité : $O(n \log(n))$ (pour le tri ou les n appel la fonction d'extraction d'un tas).

- Opérateurs en OCaml/Python :

	OCaml	Python
test d'égalité	`=`	`==`
test de différence	`<>`	`!=`
division euclidienne	`/`	`//`
modulo	`mod`	`%`
et	`&&`	`and`
ou	`||`	`or`

- Références :

	OCaml
Définition	`let r = ref ...`
Accéder à la valeur	`!r`
Modifier la valeur	`r := ...`

- Décomposer un `tuple` : `let a, b, ... = p`.
- Définir une fonction : `let f x y = ...` ou `fun x y -> ...`
- On créé souvent une liste avec une fonction récursive :

```
let rec range n = (* renvoie [n - 1; ...; 1; 0] *)
    if n = 0 then []
    else (n-1)::range (n - 1)
```

On peut éventuellement utiliser une référence sur une liste :

```
let range n =
    let l = ref [] in (* moins idiomatique *)
    for i = n - 1 downto 0 do
        l := i::!l
    done;
    !l
```

- **Impossible de modifier une liste** `l` : `e::l` renvoie une **nouvelle liste** mais ne modifie pas `l`.
  `let l = [] in ...` ne sert à rien.
- **Impossible d'écrire** `l.(i)` pour une liste `l` : il faut utiliser une fonction récursive pour parcourir une liste.
  Exemple :

```
let rec appartient e l = match l with
    | [] -> false
    | t::q -> e = t || appartient e q
```

- Chaque cas d'un `match` sert à **définir de nouvelles variables**, et ne permet pas de comparer des valeurs.
  Exemple : Dans `appartient`, il ne faut pas écrire `| e::q -> ...` (ceci remplacerait le `e` en argument).
  Pour tester l'égalité : `| t::q -> if t = e then ...` ou `| t::q when t = e -> ...`.
- Les indices d'un tableau à n éléments vont de 0 à $n-1$:

```
let appartient e t =
    let r = ref false in
    for i = 0 to  n - 1 do
        if t.(i) = e then r := true
    done;
    !r
```

Cette fonction est de type `'a -> 'a array -> bool`.

- **Impossible de renvoyer une valeur à l'intérieur d'une boucle**:

```
let appartient e t =
    for i = 0 to n - 1 do
        if t.(i) = e then true (* NE MARCHE PAS !!! *)
    done;
    false
```

- Types union et enregistrement :

```
(* type union : l'un OU l'autre *)
type 'a option = None | Some of 'a
(* on utilise match (et fonction récursive)
   sur les types union *)
let add x  y = match x, y with
    | None, _ | _, None -> None
    | Some x1, Some y1 -> Some (x1 + y1)

(* type enregistrement : l'un ET l'autre *)
type fraction = { num : int; denum : int }
let f = { num = 1; denum = 4 }
f.denum (* donne 4 *)
```

- Quelques fonctions utiles (avec les équivalents `Array.map`...):
  - `Array.make_matrix n p e` renvoie une matrice $n \times p$ dont chaque élément est `e`
  - `List.iter f l` appelle `f` sur chaque élément de `l`
  - `List.map f [a1; ...; an]` renvoie `[f a1; ...; f an]`
  - `List.filter f l` renvoie la liste des éléments `e` tels que `f e`
  - `List.exists f l` et `List.for_all f l`.

Type	Exemple	Obtenir valeur	Modification	Taille	Création
`array`	`[|1; 2|] : int array`	`t.(i)`	`t.(i) <- ...`	`Array.length` (en $O(1)$)	`Array.make n e` (en $O(n)$)
`string`	`"abc" : string`	`s.[i]`		`String.make n e` (en $O(n)$)	`String.make n e`
`list`	`[1; 2] : int list`	Fonction récursive		`List.length` (en $O(n)$)	
`tuple`	`(1, "abc", [1; 2]) : int*string*int list`	`let a, b, c = t`			

Contrairement aux `list`, `array` et `string`, un `tuple` (n-uplet) peut contenir des types différents.

- Définitions :

	Signification	Exemple
Alphabet	Ensemble fini de lettres	$\Sigma = \{a, b\}$
Mot	Suite finie de lettre	$m = abaa$
ε	Mot vide (sans lettre)	
Langage	Ensemble de mots	$L = \{\varepsilon, a, baa\}$

ε est un mot, pas une lettre.

- Opérations sur des mots $u = u_1...u_n$ et $v = v_1...v_p$:

	Définition	Exemple avec $u = ab$ et $v = cbc$
Concaténation	$uv = u_1...u_n v_1...v_p$	$uv = abcbc$
Puissance	$u^n = u...u$	$u^3 = ababab$
Taille	$\|u\| = n$	$\|u\| = 2$

Deux mots sont égaux s'ils ont la même taille et les mêmes lettres.

- Opérations sur des langages $L_1 = \{\varepsilon, ab\}$ et $L_2 = \{b, ab\}$:

	Définition	Exemple
Concaténation	$L_1L_2 = \{uv \| u \in L_1, v \in L_2\}$	$L_1L_2 = \{b, ab, abb, abab\}$
Puissance	$L^n = \{u^n \| u \in L\}$	$L_1^2 = \{\varepsilon, ab, abab\}$
Etoile	$L^* = \bigcup_{k \in \mathbb{N}} L^k$	$L_1^* = \{\varepsilon, ab, abab...\}$

Comme L_1 et L_2 sont des ensembles, on peut aussi considérer $L_1 \cup L_2$, $L_1 \cap L_2$...

- Les langages rationnels sont tous ceux qu'on peut obtenir avec les règles suivantes :
  - Un langage fini est rationnel
  - L_1 et L_2 rationnels $\implies$ $L_1 \cup L_2$ rationnel
  - L_1 et L_2 rationnels $\implies$ L_1L_2 rationnel
  - L rationnel $\implies$ L^* rationnel

  Exemples : Un alphabet Σ est toujours rationnel car fini. Σ^* est rationnel car est l'étoile du langage rationnel Σ.

- Une expression rationnelle est une suite de symboles contenant : lettres, $\emptyset$, ε, $|$ (union, parfois notée $+$), *.
  À chaque expression rationnelle e on associe un langage $L(e)$.

  Exemple : Le langage de l'expression rationnelle $e = a^*b \mid \varepsilon$ est $L(e) = (\{a\}^*\{b\}) \cup \{\varepsilon\}$.

- L rationnel $\iff$ $\exists$ une expression rationnelle de langage L.

- Définition possible d'expression rationnelle en OCaml :

```
type 'a regexp =
    | Vide | Epsilon | L of 'a
    | Union of 'a regexp * 'a regexp
    | Concat of 'a regexp * 'a regexp
    | Etoile of 'a regexp

(* définition de e ci-dessus *)
let e = Union(Concat(Etoile(a), b), Epsilon)
```

  Exemple : déterminer si ε appartient au langage de e.

```
let rec has_eps e = match e with
    | Vide | L _ -> false
    | Epsilon | Etoile _ -> true
    | Union(e1, e2) -> has_eps e1 || has_eps e2
    | Concat(e1, e2) -> has_eps e1 && has_eps e2
```

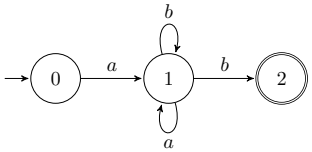
- Quelques techniques de preuve :
  - Sur des mots : récurrence sur la taille du mot.
  - Pour montrer l'égalité de deux langages : double inclusion ou suite d'équivalences.
  - Pour montrer $P(L)$ pour un langage rationnel L : par récurrence, en montrant le cas de base (si L est un langage fini) et les cas d'hérédité ($P(L_1) \wedge P(L_2) \implies P(L_1L_2)$, $P(L_1) \wedge P(L_2) \implies P(L_1 \cup L_2)$, $P(L) \implies P(L^*)$).
  - Pour montrer $P(e)$ pour une expression rationnelle e : par récurrence, en montrant les cas de base ($P(\emptyset)$, $P(\varepsilon)$, $P(a)$, $\forall a \in \Sigma$) et les cas d'hérédité ($P(e_1)$ et $P(e_2) \implies P(e_1e_2)$ et $P(e_1|e_2)$, $P(e) \implies P(e^*)$).

  Exemple : Le miroir d'un mot $u = u_1...u_n$ est $\widetilde{u} = u_n...u_1$ et le miroir d'un langage L est $\widetilde{L} = \{\widetilde{u} | u \in L\}$.
  Montrons : L rationnel $\implies$ $\widetilde{L}$ rationnel.
  On pourrait le montrer par récurrence, mais il est peut-être plus simple de définir une fonction $f(e)$ qui à une expression rationnelle e associe une expression rationnelle pour le miroir de $L(e)$:
  - $f(\emptyset) = \emptyset$, $f(\varepsilon) = \varepsilon$ et $\forall a \in \Sigma, f(a) = a$.
  - $f(e_1e_2) = f(e_2)f(e_1)$ (le miroir de uv est $\widetilde{v}\widetilde{u}$).
  - $f(e_1|e_2) = f(e_1)|f(e_2)$.
  - $f(e_1^*) = f(e_1)^*$.

  On a bien défini une fonction f telle que, pour toute expression rationnelle e, $f(e)$ est une expression rationnelle de $\widetilde{L(e)}$. Donc le miroir d'un langage rationnel est rationnel.

- Un **automate** (non dét.) est un 5-uplet $A = (\Sigma, Q, I, F, E)$ où :
  - Σ est un alphabet
  - Q est un ensemble fini d'**états**
  - $I \in Q$ est un ensemble d'**états initiaux**
  - $F \subseteq Q$ est un ensemble d'**états acceptants** (ou **états finaux**)
  - $E \subseteq Q \times \Sigma \times Q$ est un ensemble de **transitions** On peut remplacer l'ensemble E de transitions par une **fonction de transition** $\delta : Q \times \Sigma \longrightarrow \mathcal{P}(Q)$
  - Un chemin dans A est **acceptant** s'il part d'un état initial pour aller dans un état final.
  - Un mot est **accepté** par A s'il est l'étiquette d'un chemin acceptant.
  - Le **langage** $L(A)$ **accepté** (ou **reconnu**) par A est l'ensemble des mots acceptés par A.

  Exemple : le langage $a(a+b)^*b$ est reconnaissable, car reconnu par l'automate ci-dessous.



- Pour déterminer algorithmiquement si un automate A accepte un mot $u = u_1...u_n$, on peut calculer de proche en proche $Q_0 = I$, $Q_1 =$ états accessibles depuis Q_0 avec la lettre u_1, $Q_2 =$ états accessibles depuis Q_0 avec la lettre u_2... et regarder si Q_n contient un état final.

- Soit $A = (\Sigma, Q, I, F, E)$ un automate :
  - A est **complet** si : $\forall q \in Q,\ \forall a \in \Sigma,\ \exists (q, a, q') \in E$
  - Un automate $(\Sigma, Q, \{q_i\}, F, E)$ est **déterministe** si :
    1. Il n'y a qu'un seul état initial q_i.
    2. $(q, a, q_1) \in E \wedge (q, a, q_2) \in E \implies q_1 = q_2$: il y a au plus une transition possible en lisant une lettre depuis un état
  - Un automate déterministe et complet possède une unique transition possible depuis un état en lisant une lettre.
    La fonction de transition est alors de la forme $\delta : Q \times \Sigma \longrightarrow Q$.
    On peut alors étendre $\delta : Q \times \Sigma \longrightarrow Q$ en une fonction de transition sur les mots $\delta^* : Q \times \Sigma^* \longrightarrow Q$ définie par :
    * $\delta^*(q, \varepsilon) = q$
    * Si $u = av$, $\delta^*(q, av) = \delta^*(\delta(q, a), v)$

    On a alors $u \in L(A) \iff \delta^*(q_i, u) \in F$.

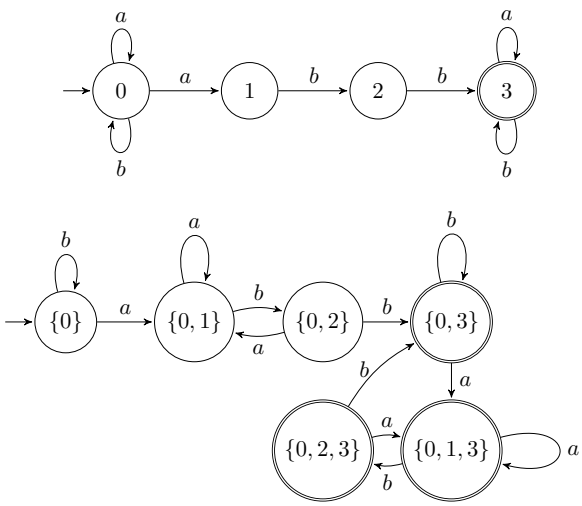
- Deux automates sont **équivalents** s'ils ont le même langage.

- Soit A un automate. Alors A est équivalent à un automate déterministe complet.

  Preuve : Utilise l'automate des parties $A' = (\Sigma, \mathcal{P}(Q), \{I\}, F', \delta')$ où $F' = \{X \subseteq Q \mid X \cap F \neq \emptyset\}$

  Remarque : Si on veut juste un automate complet (pas forcément déterministe), on peut ajouter un état poubelle vers lequel on redirige toutes les transitions manquantes. Dans l'automate des parties, cet état poubelle correspond à $\emptyset$.

  Exemple : Un automate A avec son déterminisé A'.



- Soit L un langage reconnaissable. Alors $\overline{L}$ $(= \Sigma^* \backslash L)$ est reconnaissable.

  Preuve : Soit $A = (\Sigma, Q, q_i, F, \delta)$ un automate déterministe complet reconnaissant L. Alors $A' = (\Sigma, Q, q_i, Q \backslash F, \delta)$ (on inverse états finaux et non-finaux) reconnaît $\overline{L}$.

- Soient L_1 et L_2 des langages reconnaissables. Alors :
  - $L_1 \cap L_2$ est reconnaissable.
  - $L_1 \cup L_2$ est reconnaissable.
  - $L_1 \backslash L_2$ est reconnaissable.

  Preuve : Soient $A_1 = (Q_1, q_1, F_1, \delta_1)$ et $A_2 = (Q_2, q_2, F_2, \delta_2)$ des automates finis déterministes complets reconnaissants L_1 et L_2. Soit $A = (Q_1 \times Q_2, (q_1, q_2), F, \delta)$ (**automate produit**) où :
  - $F = F_1 \times F_2$: A reconnait $L_1 \cap L_2$.
  - $F = \{(q_1, q_2) \mid q_1 \in F_1 \text{ ou } q_2 \in F_2\}$: A reconnait $L_1 \cup L_2$.
  - $F = \{(q_1, q_2) \mid q_1 \in F_1 \text{ et } q_2 \notin F_2\}$: A reconnait $L_1 \setminus L_2$.

  Remarque : Comme l'ensemble des langages reconnaissables est égal à l'ensemble des langages rationnels, l'ensemble des langages rationnels est aussi stable par complémentaire, intersection et différence.
  Il n'y a pas de stabilité par inclusion (L rationnel et $L' \subseteq L$ n'implique pas forcément L' rationnel).

- (**Lemme de l'étoile**) Soit L un langage reconnaissable par un automate à n états.
  Si $u \in L$ et $|u| \geq n$ alors il existe des mots x, y, z tels que :
  - $u = xyz$
  - $|xy| \leq n$
  - $y \neq \varepsilon$
  - $xy^*z \subseteq L$ (c'est-à-dire : $\forall k \in \mathbb{N},\ xy^kz \in L$)

<u>Application</u> : $L_1 = \{a^n b^n \mid n \in \mathbb{N}\}$ n'est pas reconnaissable.
<u>Preuve</u> : Supposons L_1 reconnaissable par un automate à n états. Soit $u = a^n b^n$. Clairement, $u \in L_1$ et $|u| \geq n$. D'après le lemme de l'étoile : il existe x, y, z tels que $u = xyz$, $|xy| \leq n$, $y \neq \varepsilon$ et $xy^*z \subseteq L$.

Comme $|xy| \leq n$, x et y ne contiennent que des a : $x = a^i$ et $y = a^j$. Comme $y \neq \varepsilon$, $j > 0$.
En prenant $k = 0$: $xy^0z = xz = a^ib^n \notin L_1$: absurde.

Automate de Glushkov : rationnel $\implies$ reconnaissable

- Une expression rationnelle est **linéaire** si chaque lettre y apparaît au plus une fois : $a(d+c)^*b$ est linéaire mais pas $ac(a+b)$.

- Soit L un langage. On définit :
  - $P(L) = \{a \in \Sigma \mid a\Sigma^* \cap L \neq \emptyset\}$ (premières lettres des mots de L)
  - $S(L) = \{a \in \Sigma \mid \Sigma^* a \cap L \neq \emptyset\}$ (dernières lettres des mots de L)
  - $F(L) = \{u \in \Sigma^2 \mid \Sigma^* u \Sigma^* \cap L \neq \emptyset\}$ (facteurs de longueur 2 des mots de L)
  - L est **local** si, pour tout mot $u = u_1 u_2 ... u_n \neq \varepsilon$:

$$u \in L \iff u_1 \in P(L) \wedge u_n \in S(L) \wedge \forall k, u_k u_{k+1} \in F(L)$$

  Il suffit donc de regarder la première lettre lettre, la dernière lettre et les facteurs de taille 2 pour savoir si un mot appartient à un langage local.
  Remarques :
  * $\implies$ est toujours vrai donc il suffit de prouver $\impliedby$.
  * Définition équivalente :

$$L \text{ local} \iff L \setminus \{\varepsilon\} = (P(L) \cap S(L)) \setminus N(L)$$

  où $N(L) = \Sigma^2 \setminus F(L)$.

  Exemples :
  - Si $L_2 = (ab)^*$ alors $P(L_2) = \{a\}$, $S(L_2) = \{b\}$ et $F(L_2) = \{ab, ba\}$. De plus si $u = u_1 u_2 ... u_n \neq \varepsilon$ avec $u_1 \in P(L), u_n \in S(L)$, et $\forall k, u_k u_{k+1} \in F(L)$ alors $u_1 = a$, $u_n = b$ et on montre (par récurrence) que $u = abab...ab \in Ł_2$. Donc L_2 est local.
  - Si $L_3 = a^* + (ab)^*$ alors $P(L_3) = \{a\}$, $S(L_3) = \{a, b\}$, $F(L_3) = \{aa, ab, ba\}$. Soit $u = aab$. La première lettre de u est dans $P(L_3)$, la dernière dans $S(L_3)$ et les facteurs de u sont aa et ba qui appartiennent à $F(L_3)$. Mais $u \notin L_3$, ce qui montre que L_3 n'est pas local.

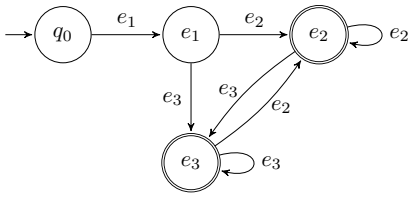
- Un automate déterministe (Σ, Q, q_0, F, E) est **local** si toutes les transitions étiquetées par une même lettre aboutissent au même état : $(q_1, a, q_2) \in E \wedge (q_3, a, q_4) \in E \implies q_2 = q_4$

- Un langage local L est reconnu par un automate local.

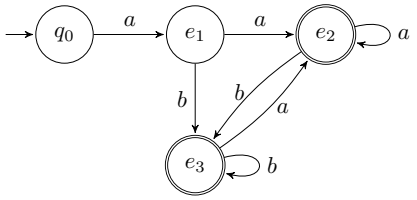
  Preuve : L est reconnu par (Σ, Q, q_0, F, E) où :
  - $Q = \Sigma \cup \{q_0\}$: un état correspond à la dernière lettre lue
  - $F = S(L)$ si $\varepsilon \notin L$, sinon $F = S(L) \cup \{q_0\}$.
  - $E = \{(q_0, a, a) \mid a \in P(L)\} \cup \{(a, b, b) \mid ab \in F(L)\}$

- L'**algorithme de Berry-Sethi** permet de construire un automate à partir d'une expression rationnelle e.

  Exemple avec $e = a(a+b)^*$:
  1. On linéarise e en e', en remplaçant chaque occurrence de lettre dans e par une nouvelle lettre : $e' = e_1(e_2 + e_3)^*$
  2. On peut montrer que $L(e')$ est un langage local.
  3. Un langage local est reconnu par l'automate local $A = (\Sigma, Q, q_0, F, E)$



  4. On fait le remplacement inverse de 1. sur les transitions de A pour obtenir un automate reconnaissant $L(e)$:



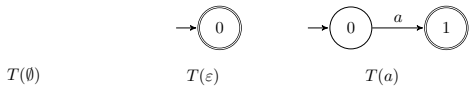
Automate de Thompson : rationnel $\implies$ reconnaissable

- Une ε-transition est une transition étiquetée par ε.

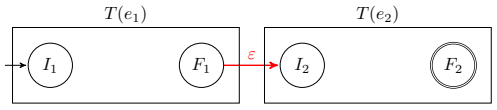
- Un automate avec ε-transitions est équivalent à un automate sans ε-transitions.

  Preuve : Si $A = (\Sigma, Q, I, F, \delta)$ est un automate avec ε-transitions, on définit $A' = (\Sigma, Q, I', F, \delta')$ où :
  - I' est l'ensemble des états atteignables depuis un état de I en utilisant uniquement des ε-transitions.
  - $\delta'(q, a)$ est l'ensemble des états q' tel qu'il existe un chemin de q à q' dans A étiqueté par un a et un nombre quelconque de ε (ce qui peut être trouvé par un parcours de graphe).

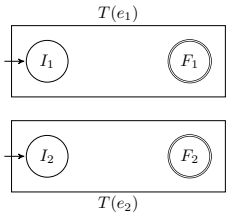
- L'**automate de Thompson** est une autre façon de convertir une expression rationnelle en automate, récursivement :
  - Cas de base :



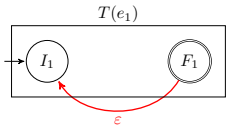
  - $T(e_1 e_2)$: ajout d'une ε-transition depuis chaque état final de $T(e_1)$ vers chaque état initial de $T(e_2)$.



  - $T(e_1 | e_2)$: union des états initiaux et des états finaux.



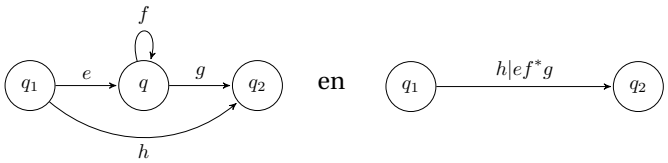
– $T(e_1^*)$: ajout d'une ε-transition depuis chaque état final vers chaque état initial.



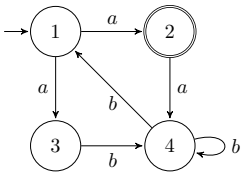
Élimination des états : reconnaissable $\Longrightarrow$ rationnel

- Tout automate est équivalent à un automate avec un unique état initial sans transition entrante et un unique état final sans transition sortante.
  Preuve : On ajoute un état initial q_i et un état final q_f et des transitions ε depuis q_i vers les états initiaux et depuis les états finaux vers q_f.

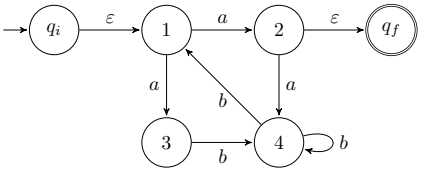
- **Méthode d'élimination des états** : On considère un automate A comme dans le point précédent. Tant que A possède au moins 3 états, on choisit un état $q \notin \{q_i, q_f\}$ et on supprime q en modifiant les transitions :



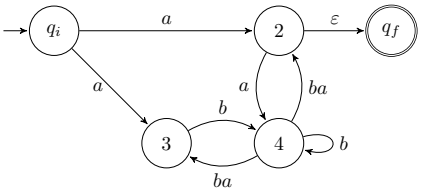
Exemple :



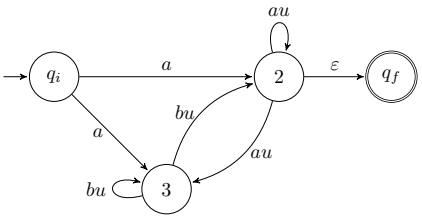
1. On commence par se ramener à un automate avec un état initial sans transition entrante et un état final sans transition sortante :



2. Suppression de l'état 1 :

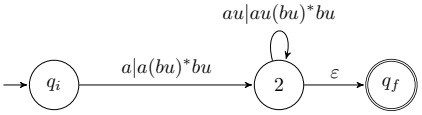


3. Suppression de l'état 4 :

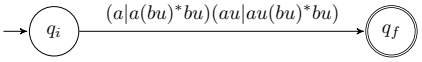


Avec $u = b^*ba$.

4. Suppression de l'état 3 :



5. Suppression de l'état 2 :



On obtient l'expression rationnelle $a|a(bu)^*bu(au|au(bu)^*bu)$, où $u = b^*ba$.

Rationnel $\Longleftrightarrow$ reconnaissable

- **Théorème de Kleene** : un langage est rationnel si et seulement si il est reconnaissable par un automate.

- Les théorèmes sur les automates s'appliquent aussi aux langages rationnels, et inversement. Notamment, les langages rationnels sont stables par union, concaténation, étoile, intersection, complémentaire, différence.